

Homework 2: EM Algorithm and Motif Finding

Bernhard Buss, Timothy Lauren Nijhuis, Carlo Rossetto and Görkem Kadir Solun

```
set.seed(42)
library(patchwork)
library(ggplot2)
```

An important driver of cancer is the mutation of tumor suppressor genes (TSG), among which we find genes coding for DNA repair proteins. The mutation and inactivation of such genes has the effect of increasing the frequency of mutations in a cell and its progeny. Suppose that we are investigating the inactivation of one specific TSG, and hypothesize that cells with this inactivation have a different probability of mutations than the ones without. The data we observe are sequencing results for certain positions of the genome, from a cell population at single cell resolution. We wish to find out from the data the effect of this TSG inactivation – how different are the probabilities of mutations with or without this inactivation? – as well as the proportion of cells with this inactivation in the population.

(4) TSG inactivation problem formulation

We model this problem with a two-component mixture model, corresponding to the cell groups with and without the TSG inactivation. Each component is parametrized by a mutation probability. Within a cell group, the probability of mutations are assumed to be the same at each position and the mutations are independent between positions.

More precisely, let $D = \{X^{(1)}, \dots, X^{(N)}\}$ be a set of sequences. Each $X^{(i)}$ has length L (we write $X^{(i)} = (X_1^{(i)}, \dots, X_L^{(i)})$), and $X_\ell^{(i)} \in \{0, 1\}$. In other words, we sequence N cells at L genomic positions, and represent the mutation status at each position by a binary variable, with 1 meaning mutated. Each sequence $X^{(i)}$ is generated either

- by the **mutated model** with parameter $\theta_1 \in (0, 1)$, where $P(X_\ell^{(i)} = 1) = \theta_1$, or
- by the **wild-type model** with parameter $\theta_2 \in (0, 1)$, where $P(X_\ell^{(i)} = 1) = \theta_2$.

The mutated model is used with probability λ_1 , and the wild-type model is used with probability $\lambda_2 = 1 - \lambda_1$.

We will learn maximum likelihood parameters $\lambda = (\lambda_1, \lambda_2)$ and $\theta = (\theta_1, \theta_2)$ using the EM algorithm.

(4a) The complete-data log-likelihood

Let us define binary hidden variables $Z^{(ik)}$, where $Z^{(ik)} = 1$ if and only if $X^{(i)}$ is generated from component k . We write down the complete-data log-likelihood $\log P(X, Z|\lambda, \theta)$ in terms of $X^{(i)}$, $Z^{(i)}$, λ , and θ :

$$\log(P(X, Z|\lambda, \theta)) = \sum_{i=1}^N \log(P(X^{(i)}, Z^{(i)}|\lambda, \theta)) = \sum_{i=1}^N \sum_{k=1}^2 Z^{(ik)} \log(\lambda_k P(X^{(i)}|\theta_k))$$

$P(X^{(i)}|\theta_k)$ follows a Bernoulli distribution with probability θ for each position up to L :

$$P(X^{(i)}|\theta_k) = \prod_{l=1}^L \theta_k^{X_l^{(i)}} (1 - \theta_k)^{(1-X_l^{(i)})}$$

Substituting, the equation becomes:

$$\begin{aligned} \log(P(X, Z|\lambda, \theta)) &= \sum_{i=1}^N \sum_{k=1}^2 Z^{(ik)} \log \left(\lambda_k \prod_{l=1}^L \theta_k^{X_l^{(i)}} (1 - \theta_k)^{(1-X_l^{(i)})} \right) \\ &= \sum_{i=1}^N \sum_{k=1}^2 Z^{(ik)} \left(\log(\lambda_k) + \sum_{l=1}^L (X_l^{(i)} \log(\theta_k) + (1 - X_l^{(i)}) \log(1 - \theta_k)) \right) \\ &= \sum_{i=1}^N \left[Z^{(i1)} \left(\log(\lambda_1) + \sum_{l=1}^L (X_l^{(i)} \log(\theta_1) + (1 - X_l^{(i)}) \log(1 - \theta_1)) \right) \right. \\ &\quad \left. + Z^{(i2)} \left(\log(\lambda_2) + \sum_{l=1}^L (X_l^{(i)} \log(\theta_2) + (1 - X_l^{(i)}) \log(1 - \theta_2)) \right) \right] \end{aligned}$$

(4b) The responsibility

Let us define the responsibility of component k for observation i as the posterior probability that $Z^{(ik)}$ is 1:

$$\gamma_{ik} := P(Z^{(ik)} = 1|X^{(i)}, \lambda, \theta).$$

Express γ_{ik} in terms of $X^{(i)}$, λ , and θ .

Using Bayes theorem:

$$P(G|C) = \frac{P(G)P(C|G)}{P(C)}$$

Resulting in:

$$P(Z^{(ik)} = 1|X^{(i)}, \lambda, \theta) = \frac{P(Z^{(ik)} = 1|\lambda)P(X^{(i)}|Z^{(ik)} = 1, \theta)}{P(X^{(i)}|\lambda, \theta)}$$

Substituting the model components prior probability:

$$P(Z^{(ik)} = 1|\lambda) = \lambda_k$$

and likelihood:

$$P(X^{(i)}|Z^{(ik)} = 1, \theta) = \prod_{l=1}^L \theta_k^{X_l^{(i)}} (1 - \theta_k)^{(1-X_l^{(i)})}$$

Resulting in:

$$= \frac{\lambda_k \prod_{l=1}^L \theta_k^{X_l^{(i)}} (1 - \theta_k)^{(1-X_l^{(i)})}}{P(X^{(i)}|\lambda, \theta)}$$

$P(X^{(i)}|\lambda, \theta)$ is the marginal probability of $X^{(i)}$, so one needs to sum over $Z^{(ik)} = 1$.

$$P(X^{(i)}|\lambda, \theta) = \sum_{j=1}^2 \lambda_j \prod_{l=1}^L \theta_j^{X_l^{(i)}} (1 - \theta_j)^{(1-X_l^{(i)})}$$

Resulting in:

$$\gamma_{ik} = \frac{\lambda_k \prod_{l=1}^L \theta_k^{X_l^{(i)}} (1 - \theta_k)^{(1-X_l^{(i)})}}{\sum_{j=1}^2 \lambda_j \prod_{l=1}^L \theta_j^{X_l^{(i)}} (1 - \theta_j)^{(1-X_l^{(i)})}}$$

(4c) The M-step update formula

We derive the M-step update formulae for parameters λ and θ .

The M-step maximise $F(q, \theta)$ with respect to θ :

$$F(q, \theta) = E_q[l_{hid}(\theta)] - E_q[\log(q(Z))]$$

Since only $E_q[l_{hid}(\theta)]$ depends on θ , $E_q[l_{hid}(\theta)]$ is maximised. For a motif mixture model, as shown in the lectures, the complete data likelihood can be further simplified to:

$$E_q[l_{hid}(\theta)] = \sum_{i,k} \gamma_{ik} \log(\lambda_k) + \sum_{i,k} \gamma_{ik} \log(P(X^{(i)}|\theta_k))$$

Maximisation with respect to the λ_k involves only the left sum term.

$$J(\lambda, \mu) = \mu \left(1 - \sum_{k=1}^2 \lambda_k \right) + \sum_{i=1}^N \sum_{k=1}^2 \gamma_{ik} \log \lambda_k$$

where μ is a Lagrange multiplier for the constraint $\sum_{k=1}^2 \lambda_k = 1$. The update formula for λ_k is obtained by setting the derivative of this objective to zero:

$$\frac{\partial J}{\partial \lambda_k} = \frac{\sum_{i=1}^N \gamma_{ik}}{\lambda_k} - \mu = 0 \quad \text{for each } k, \quad \text{and} \quad \sum_{k=1}^2 \lambda_k = 1.$$

Rearranging, we have that

$$\sum_{i=1}^N \gamma_{ik} = \mu \lambda_k, \quad \text{for each } k.$$

Summing the two equations from $k = 1$ and $k = 2$ together, and using the fact that by definition $\sum_{k=1}^2 \gamma_{ik} = 1$ for each i and $\sum_{k=1}^2 \lambda_k = 1$, we have that $\mu = N$, and where N is the total number of cells.

$$\lambda_k^{new} = \frac{\sum_{i=1}^N \gamma_{ik}}{N} = \frac{\sum_{i=1}^N \frac{\lambda_k \prod_{l=1}^L \theta_k^{X_l^{(i)}} (1-\theta_k)^{(1-X_l^{(i)})}}{\sum_{j=1}^2 \lambda_j \prod_{l=1}^L \theta_j^{X_l^{(i)}} (1-\theta_j)^{(1-X_l^{(i)})}}}{N}$$

Regarding maximising for θ_k , it is the right sum that should be maximised (because it is the sum that depends on θ_k). Since from (4a)

$$P(X^{(i)}|\theta_k) = \prod_{l=1}^L \theta_k^{X_l^{(i)}} (1-\theta_k)^{(1-X_l^{(i)})}$$

The right sum becomes

$$\begin{aligned} \sum_{i,k} \gamma_{ik} \log(P(X^{(i)}|\theta_k)) &= \sum_{i=1}^N \sum_{k=1}^2 \gamma_{ik} \log(P(X^{(i)}|\theta_k)) = \sum_{i=1}^N \sum_{k=1}^2 \gamma_{ik} \log \left(\prod_{l=1}^L \theta_k^{X_l^{(i)}} (1-\theta_k)^{(1-X_l^{(i)})} \right) = \\ &= \sum_{i=1}^N \sum_{k=1}^2 \gamma_{ik} \sum_{l=1}^L (X_l^{(i)} \log(\theta_k) + (1-X_l^{(i)}) \log(1-\theta_k)) \end{aligned}$$

For ease the expression above is set $= f$. In order to find the optimal θ the first derivative is set $= 0$.

$$f = \sum_{i=1}^N \sum_{k=1}^2 \gamma_{ik} \sum_{l=1}^L (X_l^{(i)} \log(\theta_k) + (1-X_l^{(i)}) \log(1-\theta_k))$$

When taking the derivative, the $\sum_{k=1}^2$ falls out

$$\begin{aligned} \frac{\partial f}{\partial \theta_k} &= 0 = \sum_{i=1}^N \gamma_{ik} \sum_{l=1}^L \left(\frac{X_l^{(i)}}{\theta_k} - \frac{1-X_l^{(i)}}{1-\theta_k} \right) \\ &= \sum_{i=1}^N \gamma_{ik} \sum_{l=1}^L \left(\frac{X_l^{(i)}(1-\theta_k) - \theta_k(1-X_l^{(i)})}{\theta_k(1-\theta_k)} \right) = \sum_{i=1}^N \gamma_{ik} \sum_{l=1}^L \left(\frac{X_l^{(i)} - \theta_k}{\theta_k(1-\theta_k)} \right) \end{aligned}$$

$$\begin{aligned}
&\Rightarrow \sum_{i=1}^N \gamma_{ik} \sum_{l=1}^L \left(\frac{\theta_k}{\theta_k(1-\theta_k)} \right) = \sum_{i=1}^N \gamma_{ik} \sum_{l=1}^L \left(\frac{X_l^{(i)}}{\theta_k(1-\theta_k)} \right) \\
&\Rightarrow \left(\frac{\theta_k}{\theta_k(1-\theta_k)} \right) \sum_{i=1}^N \gamma_{ik} \sum_{l=1}^L 1 = \left(\frac{1}{\theta_k(1-\theta_k)} \right) \sum_{i=1}^N \gamma_{ik} \sum_{l=1}^L X_l^{(i)} \\
&\Rightarrow \theta_k \cdot L \sum_{i=1}^N \gamma_{ik} = \sum_{i=1}^N \gamma_{ik} \sum_{l=1}^L X_l^{(i)} \\
&\Rightarrow \theta_k^{new} = \frac{\sum_{i=1}^N \gamma_{ik} \sum_{l=1}^L X_l^{(i)}}{L \sum_{i=1}^N \gamma_{ik}}
\end{aligned}$$

(5) TSG inactivation problem implementation

Suppose that we sequenced 200 cells from a tumor biopsy sample. The file ‘mutations.csv’ contains mutation calls (denoted 1 if the position is mutated, and 0 otherwise) for 100 positions of interest in the genome, for the 200 cells in the sample. We will implement the EM algorithm formulated above, to learn which cells belong to which group and the mutation probabilities of each group. Please use the R code skeleton hw2_EM_skeleton.qmd on Moodle for this question, and make sure to include the filled-out code sections in your submission.

(5a) The EM algorithm

Complete the implementation of the EM algorithm in the code skeleton. Note: Beware of numerical instability when multiplying large amount of numbers together – do additions of logarithms whenever possible.

Please write your code in the indicated blocks.

```
##' This function implements the EM algorithm for the gene mutation problem
##' @param D Data matrix of dimensions N-by-100, where N is the number of observations
##' @param k Number of cell groups
##' @param lambda Vector of mixture weights
##' @param theta Vector of probabilities of mutations
##' @param tolerance A threshold used to check convergence
mutation_EM <- function(D, k, lambda, theta, tolerance = 1e-2) {

  # observed log-likelihood
  ll_obs <- -Inf
  # difference between two iterations
  diff <- Inf
  # number of observations
```

```

N <- nrow(D)
# number of positions
L <- ncol(D)
# responsibilities
gamma <- matrix(0, nrow = k, ncol = N)
# keep track of lambda and theta during the optimization
lambda_all <- lambda
theta_all <- theta
# iteration number
t <- 1

# run the E-step and M-step until convergence
while (diff > tolerance) {

  ##### E-step #####

  ### YOUR CODE STARTS ###

  # Compute the responsibilities using log-space to avoid underflow
  # For each observation i and component j, compute
  # log(lambda_j) + sum_l [ X_l^(i) * log(theta_j) + (1 - X_l^(i)) * log(1 - theta_j) ]
  log_rho <- matrix(0, nrow = k, ncol = N)
  for (j in 1:k) {
    # Sum of X_l * log(theta_j) + (1-X_l) * log(1-theta_j) for each observation
    log_lik <- D %*% rep(log(theta[j]), L) + (1 - D) %*% rep(log(1 - theta[j]), L)
    # log(lambda_j) + log(P(X^(i) | theta_j))
    log_rho[j, ] <- log(lambda[j]) + as.vector(log_lik)
  }

  # Normalise responsibilities using log-sum-exp trick
  log_sum <- apply(log_rho, 2, function(x) {
    m <- max(x)
    m + log(sum(exp(x - m)))
  })
  for (j in 1:k) {
    gamma[j, ] <- exp(log_rho[j, ] - log_sum)
  }

  # Update observed log-likelihood
  # ll_obs_new = sum_i log( sum_j lambda_j * P(X^(i) | theta_j) )
  ll_obs_new <- sum(log_sum)
  ll_obs <- c(ll_obs, ll_obs_new)
}

```

```

# keep track of ll_obs throughout iterations

# Recompute difference between two iterations
diff <- abs(ll_obs[t] - ll_obs[t+1])

##### M-step #####

# Recompute priors (mixture weights)
# lambda_j^new = sum_i gamma_ij / N
for (j in 1:k) {
  lambda[j] <- sum(gamma[j, ]) / N
}

lambda_all <- rbind(lambda_all, lambda)
# keep track of lambda throughout iterations

# Recompute probability of mutation for each cell group
# theta_j^new = sum_i gamma_ij * sum_l X_l^i / (L * sum_i gamma_ij)
for (j in 1:k) {
  numerator <- sum(gamma[j, ] * rowSums(D))
  denominator <- L * sum(gamma[j, ])
  theta[j] <- numerator / denominator
}

theta_all <- rbind(theta_all, theta)
# keep track of theta throughout iterations

t <- t+1
}
### YOUR CODE ENDS ###

return(list(ll_obs = ll_obs, lambda = lambda, theta = theta, gamma = gamma,
           lambda_all = lambda_all, theta_all = theta_all))
}

```

(5b) Load data, initialize parameters, and run the EM algorithm.

Use the given code to read in the data stored in the file ‘mutations.csv’, and to initialize the mixture weights and the mutation probabilities . We assume a priori that the mixture weights

are uniform, and $\pi_1 > \pi_2$. Run the completed EM algorithm on this data.

```
# read the data into D
D <- read.csv("mutations.csv")
D <- as.matrix(D)
# check the dimension of D
all(dim(D) == c(200, 100))
```

```
[1] TRUE
```

```
# number of cell groups
k <- 2
# mixture weights (a vector of length k)
lambda <- c(0.5, 0.5)
# probabilities of mutation (having a 1 in the data) (a vector of length k)
theta <- c(0.6, 0.4)

# run the EM algorithm
res <- mutation_EM(D, k, lambda, theta)
```

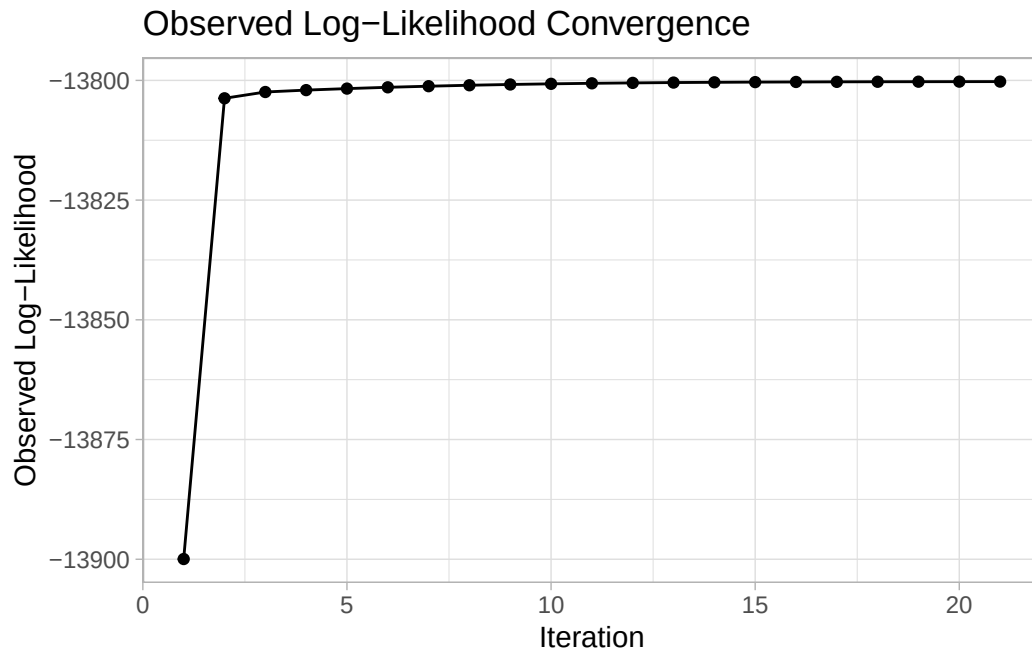
(5c) Plotting convergence.

Plot the following quantities (y-axis) as a function of the number of iterations (x-axis) in three separate plots: • the observed log-likelihood (recall that this is computed by marginalizing out the hidden variables) • the mutation probabilities π_1 and π_2 , and • the mixture weights π_1 and π_2 .

Observed log-likelihood throughout iterations:

```
# Remove the initial -Inf for plotting
ll_data <- data.frame(
  Iteration = 1:(length(res$ll_obs) - 1),
  LogLikelihood = res$ll_obs[-1]
)

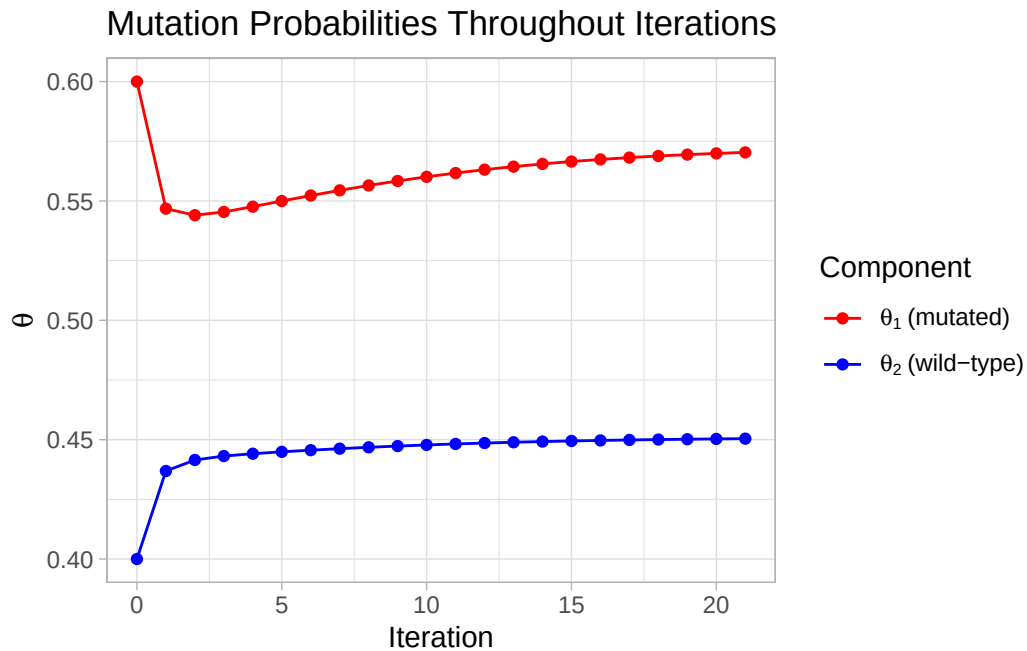
ggplot(ll_data, aes(x = Iteration, y = LogLikelihood)) +
  geom_line() +
  geom_point(size = 1.5) +
  labs(title = "Observed Log-Likelihood Convergence",
       x = "Iteration", y = "Observed Log-Likelihood") +
  theme_light()
```



Probabilities of mutation throughout iterations:

```
theta_data <- data.frame(
  Iteration = 0:(nrow(res$theta_all) - 1),
  Theta1 = res$theta_all[, 1],
  Theta2 = res$theta_all[, 2]
)
theta_long <- rbind(
  data.frame(Iteration = theta_data$Iteration, Component = "Theta1", Theta = theta_data$Theta1),
  data.frame(Iteration = theta_data$Iteration, Component = "Theta2", Theta = theta_data$Theta2)
)

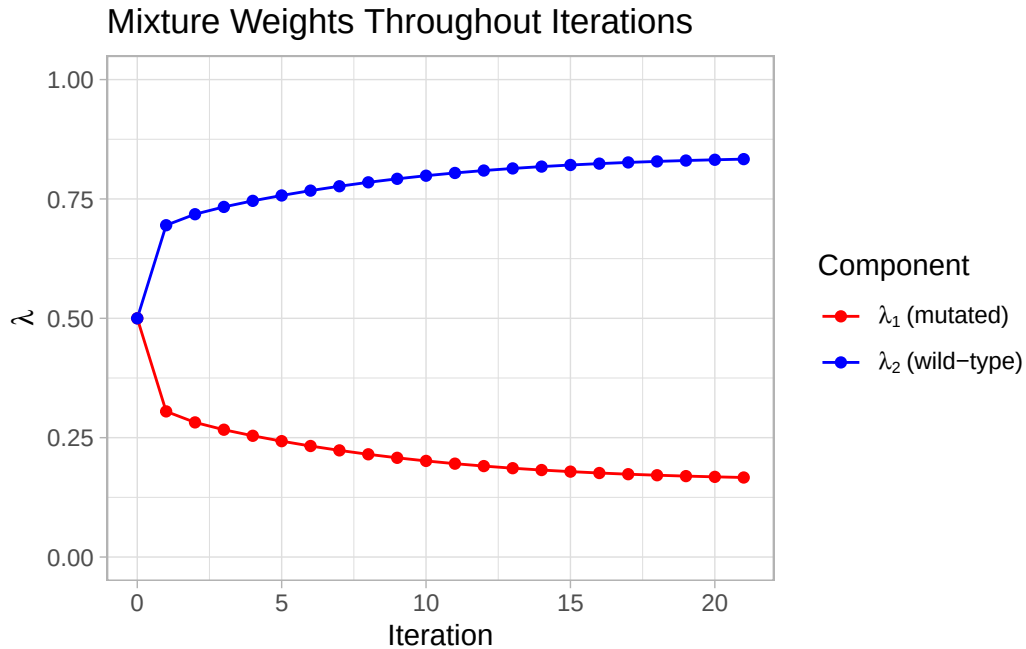
ggplot(theta_long, aes(x = Iteration, y = Theta, color = Component)) +
  geom_line() +
  geom_point(size = 1.5) +
  labs(title = "Mutation Probabilities Throughout Iterations",
       x = "Iteration", y = expression(theta)) +
  scale_color_manual(values = c("Theta1" = "red", "Theta2" = "blue"),
                    labels = c(expression(theta[1]~"(mutated)"),
                               expression(theta[2]~"(wild-type)"))) +
  theme_light()
```



Mixture weights throughout iterations:

```
lambda_data <- data.frame(
  Iteration = 0:(nrow(res$lambda_all) - 1),
  Lambda1 = res$lambda_all[, 1],
  Lambda2 = res$lambda_all[, 2]
)
lambda_long <- rbind(
  data.frame(Iteration = lambda_data$Iteration, Component = "Lambda1", Lambda = lambda_data$Lambda1),
  data.frame(Iteration = lambda_data$Iteration, Component = "Lambda2", Lambda = lambda_data$Lambda2)
)

ggplot(lambda_long, aes(x = Iteration, y = Lambda, color = Component)) +
  geom_line() +
  geom_point(size = 1.5) +
  labs(title = "Mixture Weights Throughout Iterations",
       x = "Iteration", y = expression(lambda)) +
  scale_color_manual(values = c("Lambda1" = "red", "Lambda2" = "blue"),
                    labels = c(expression(lambda[1]~"(mutated)",
                                         expression(lambda[2]~"(wild-type)")))) +
  ylim(0, 1) +
  theme_light()
```



(5d) Responsibilities at final iteration:

Plot a heatmap of the responsibilities at the final step, where columns are sequences, rows are components, and values indicate the responsibility of component k for $X^{(i)}$. For each cell, we assign it to the component with higher probability. How many cells belong to the mutated group, and how many cells belong to the wild-type group?

Heatmap of responsibilities:

```
# gamma is k x N; rows = components, columns = sequences
# Sort cells by responsibility to the mutated component for clearer visualization
cell_order <- order(res$gamma[1, ], decreasing = TRUE)
gamma_sorted <- res$gamma[, cell_order]

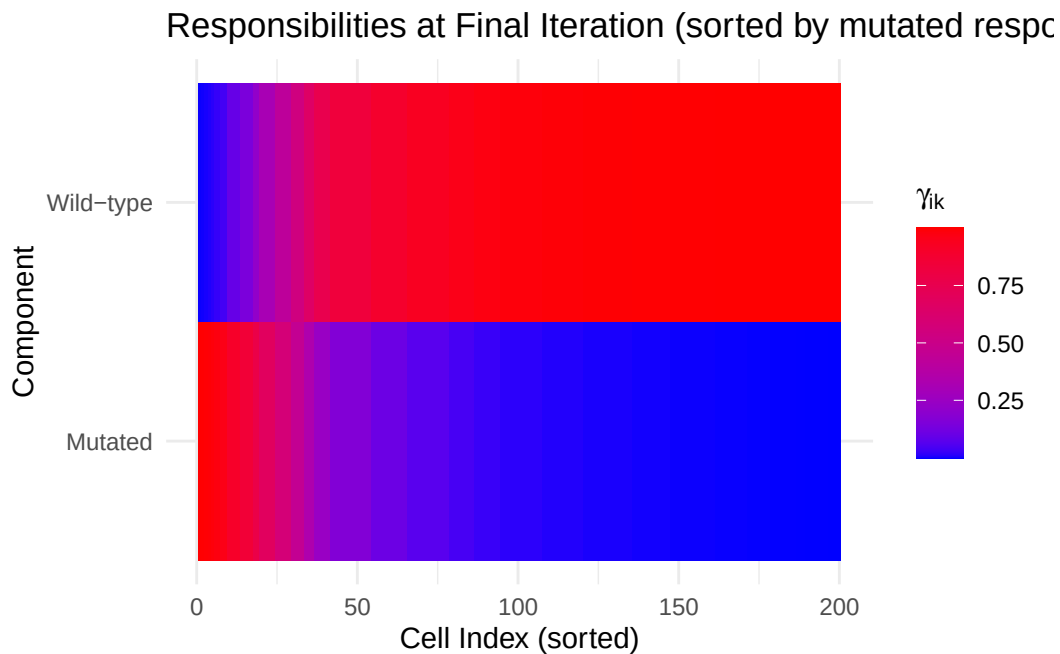
resp_data <- rbind(
  data.frame(Component = "Mutated", Cell = 1:ncol(gamma_sorted),
    Responsibility = gamma_sorted[1, ]),
  data.frame(Component = "Wild-type", Cell = 1:ncol(gamma_sorted),
    Responsibility = gamma_sorted[2, ])
)

ggplot(resp_data, aes(x = Cell, y = Component, fill = Responsibility)) +
  geom_tile() +
```

```

scale_fill_gradient(low = "blue", high = "red",
                    name = expression(gamma[ik])) +
labs(title = "Responsibilities at Final Iteration (sorted by mutated responsibility)",
     x = "Cell Index (sorted)", y = "Component") +
theme_minimal()

```

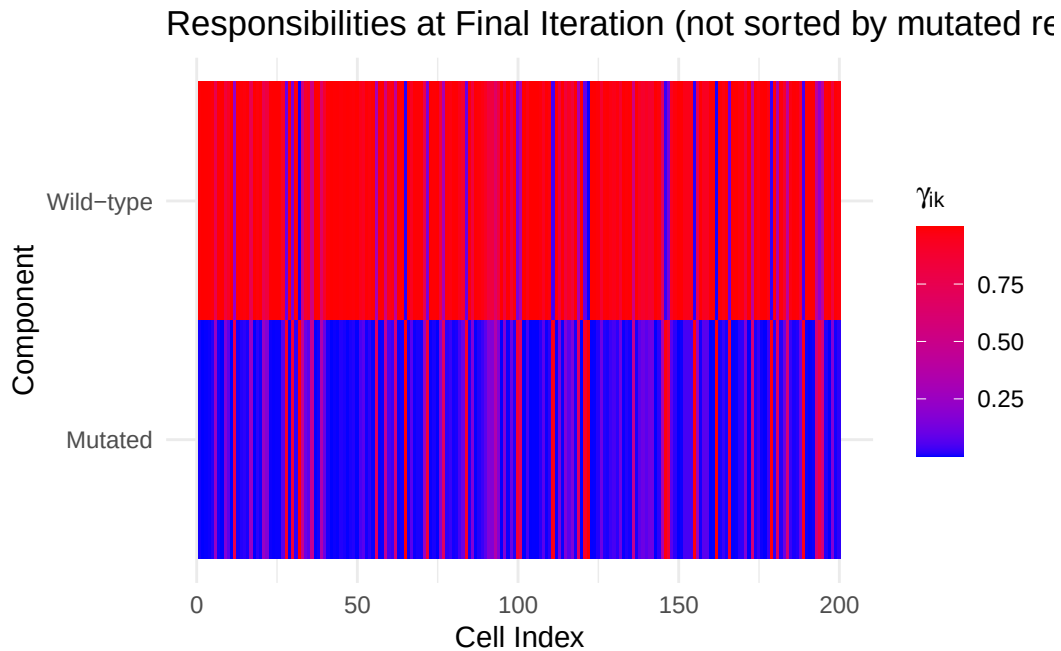


```

# gamma is k x N; rows = components, columns = sequences
resp_data <- rbind(
  data.frame(Component = "Mutated", Cell = 1:ncol(gamma_sorted),
             Responsibility = res$gamma[1, ]),
  data.frame(Component = "Wild-type", Cell = 1:ncol(gamma_sorted),
             Responsibility = res$gamma[2, ])
)

ggplot(resp_data, aes(x = Cell, y = Component, fill = Responsibility)) +
  geom_tile() +
  scale_fill_gradient(low = "blue", high = "red",
                    name = expression(gamma[ik])) +
labs(title = "Responsibilities at Final Iteration (not sorted by mutated responsibility)",
     x = "Cell Index", y = "Component") +
theme_minimal()

```



How many cells belong to the mutated group, and how many cells belong to the wild-type group?

```
# Assign each cell to the component with the highest responsibility
n_mutated <- sum(res$gamma[1, ] > 0.5)
n_wildtype <- sum(res$gamma[1, ] <= 0.5)
cat("Number of cells in the mutated group:", n_mutated, "\n")
```

Number of cells in the mutated group: 29

```
cat("Number of cells in the wild-type group:", n_wildtype, "\n")
```

Number of cells in the wild-type group: 171